

Homework 1

Dávid Straka

October 6, 2023

Exercise 1

The given sum can be numerically approximated by $\frac{1570873}{500000} = 3.141746 \approx \pi$ for $n = 10000000$.

This convergence happens because

$$\mathbb{E} \left[\frac{4}{n} \sum_{i=1}^n \mathbb{1} \left\{ (U_1^{(i)})^2 + (U_2^{(i)})^2 \leq 1 \right\} \right] = \frac{4}{n} \sum_{i=1}^n P \left((U_1^{(i)})^2 + (U_2^{(i)})^2 \leq 1 \right) = \frac{4}{n} n \frac{\pi}{4} = \pi.$$

Notice that

$$P \left((U_1^{(i)})^2 + (U_2^{(i)})^2 \leq 1 \right) = \frac{C}{A} = \frac{\frac{\pi}{2}}{2} = \frac{\pi}{4},$$

where A is the area of the rectangle $U[-1, 1] \times U[0, 1]$ and C is the area of the semi-circle $(U_1^{(i)})^2 + (U_2^{(i)})^2 \leq 1$.

Exercise 2

By Inversion, the inverse

$$F^{\leftarrow}(U) = (-\log(U))^{\frac{1}{\beta}}$$

of the Weibull CDF, where $U \sim U(0, 1)$, generates Weibull-distributed random variables.

Check the routine and histogram in Mathematica notebook below.

Exercise 3

Since $0 \leq \bar{H}(x) \leq 1$, $f(x) = cg(x)\bar{H}(x) \leq cg(x)$. Notice $c \geq 1$, otherwise, $f(x)$ is not a density. Hence, by A-R, we take $X = Y$ iff

$$U \leq \frac{cg(Y)\bar{H}(Y)}{cg(Y)} = \bar{H}(Y) = P(Z > Y) = P(Z \geq Y). \quad (1)$$

If we sample $Z \geq Y$, then $U \leq 1$. Hence, we accept $X = Y$ by Inequality 1. If we sample $Z < Y$, then $U \leq 0$, which violated Inequality 1 because $P(U = 0) = 0$. Hence, we reject $X = Y$. This means that we accept Y as X iff the sampled Z and Y satisfy $Z \geq Y$.

Exercise 4

Rewrite the PDF of the inverse Gaussian as

$$f(x) = \frac{e}{\sqrt{2\pi}} x^{-\frac{3}{2}} \exp\left(-\frac{1}{2x}\right) \exp\left(-\frac{x}{2}\right) = z(x) \exp\left(-\frac{x}{2}\right)$$

where

$$z(x) = \frac{e}{\sqrt{2\pi}} x^{-\frac{3}{2}} \exp\left(-\frac{1}{2x}\right).$$

We use Mathematica to find the maximum value $3\sqrt{\frac{3}{2e\pi}}$ of $z(x)$. Hence,

$$f(x) \leq \left(3\sqrt{\frac{3}{2e\pi}}\right) \exp\left(-\frac{x}{2}\right).$$

Use $\text{Exp}(\frac{1}{2})$ to bound $f(x)$ as

$$f(x) \leq cg(x)$$

where $c = 3\sqrt{\frac{6}{e\pi}} \approx 2.51463$ and $g(x)$ is the PDF of $\text{Exp}(\frac{1}{2})$.

Check Mathematica notebook below for the routine for sampling from the inverse Gaussian.

The given formulas give that $\mathbf{E}X = 1$ and $\text{Var}X = 1$. Hence, $\mathbf{E}X^2 = \text{Var}X + (\mathbf{E}X)^2 = 2$. The generated sample gives that $\bar{X}_n \approx 0.994$, $\bar{X}_n^2 \approx 1.915$, and $S_X^2 \approx 0.927$.

Since $\text{Var}X = 1 < \infty$, by CLT, the CI of $\bar{X}_n$ with confidence level $\alpha = 0.95$ and $n = 10000$ is $(\mathbf{E}X - q_{0.95} \frac{\text{Var}X}{\sqrt{n}}, \mathbf{E}X + q_{0.95} \frac{\text{Var}X}{\sqrt{n}}) = (0.9804, 1.0194)$. Notice that $\bar{X}_n \approx 0.994 \in (0.9804, 1.0194)$.

Check Mathematica notebook below to see construction of the confidence intervals. Notice that

$$\mathbf{E}X^2 = 2 \in (1.32, 2.65), \text{ and}$$

$$\text{Var}X = 1 \in (0.97, 1.02).$$

This confirms the routine via confidence intervals.

Exercise 5

The mean of Weibull distribution is given by $\lambda\Gamma(1 + \frac{1}{k})$. For $k = 1$, the mean is λ . By inversion, we can generate the Weibull distribution from $-\lambda\log(U)$ with $U \sim U(0, 1)$.

I used the Box–Muller method to generate the standard normal distribution (book p.42).

The idea was to create a new kind of fast departure that happened to store the information of the number of available berths. Therefore, I left the variables related to berths out as they had no effect on the code.

The average waiting time of ships in the harbor until they are unloaded is 0.874 days and the longest waiting time is 2.15 days. I ran the simulation $n = 100$ times to check result consistency. The standard deviation after n simulations was 0.095 for the average waiting time of ships and 0.42 for the longest waiting time. I would judge that as rather too high to deem the estimates reliable.

Homeworld 1

Exercise 1

```
In[1]:= (* Func draws from the uniform distribution on [a,b],
        using a standard uniform sample given by the function RandomReal[] *)
        drawUniformNumber[a_, b_] := a + (b - a) * RandomReal[];
        reelle Zufallszahl

In[2]:= (* Func draws n samples of U=(U1, U2) w/ U1~U[-1,1], U2~U[0,1] *)
        drawTwoSamples[n_] :=
        Table[{drawUniformNumber[-1, 1], drawUniformNumber[0, 1]}, {i, 1, n}];
        Tabelle

In[3]:= (* Computes numerically given sum of n summands *)
        n = 100 000;
        U = drawTwoSamples[n];
        4 / n Sum[If[U[[i]][[1]]^2 + U[[i]][[2]]^2 ≤ 1, 1, 0], {i, 1, n}]
        s... wenn

Out[5]= 78 477
        25 000
```

Exercise 2

```
In[6]:= (* Func draws from the Weibull distr. w/ shape param. beta_ and scale param. 1 *)
        drawWeibullNumber[beta_] := (-Log[RandomReal[]])^(1/beta)
        L... reelle Zufallszahl
```

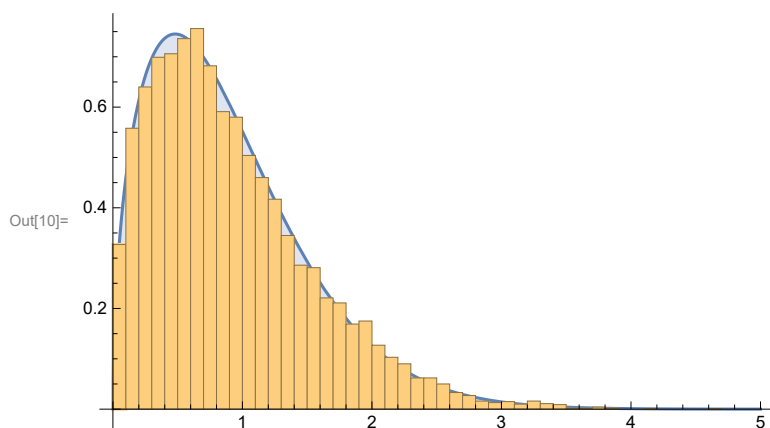
```

In[7]:= (* Plots a histogram of n simulated values of the Weibull
         distr. against the PDF of the Weibull distr. w/ shape param. beta *)
         [Wahrscheinlichkeitsdichtefunktion]

beta = 1.5;
n = 10000;
data = Table[drawWeibullNumber[beta], n];
         [Tabelle]

Show[
         [zeige an]
         Plot[PDF[WeibullDistribution[beta, 1], x] // Evaluate, {x, 0, 5}, Filling -> Axis],
         [stell... [W... [Weibull-Verteilung] [werte aus] [Füllung] [Achse]
         Histogram[data, Automatic, "PDF"]]
         [Histogramm] [automatisch] [Wahrscheinlichkeitsdichtefunktion]

```



Exercise 4

```

In[11]:= (* Find maximum of func z(x) *)
         [finde]

z[x_] := E / Sqrt[2 * Pi] * x^(-3/2) * Exp[-1 / (2 * x)];
         [... [Quadratw... [Kreiszahl π] [Exponentialfunktion]

c = Maximize[{z[x], x > 0}, x] [[1]]
         [maximiere]

```

Out[12]= $3 \sqrt{\frac{3}{2 e \pi}}$

```

In[13]:= (* Generates n samples from inverse Gaussian using A-R *)
f[x_] := z[x] Exp[-x/2];
           |Exponentialfunktion
g[x_] := (1/2) * Exp[-x/2];
           |Exponentialfunktion

n = 10000;
X = {};

While[Length[X] < n,
  |solange |Länge

  (* samples from E(1/2) using Inversion *)
  |Exponentialkonstante E
  Y = -2 * Log[RandomReal[]];
  |L... |reelle Zufallszahl
  If[RandomReal[] <= f[Y] / (2 * c * g[Y]), AppendTo[X, Y]; ] ;
  |w... |reelle Zufallszahl |hänge an bei
  ];

In[18]:= (* Constructs confidence intervals *)
XSquared = X^2;
meanX = Mean[X];
           |arithmetisches Mittel
meanXSquared = Mean[XSquared];
           |arithmetisches Mittel
varX = Variance[X];
           |Varianz
varXSquared = Variance[XSquared];
           |Varianz

Print["Sample mean of X is: ", meanX]
|gib aus
Print["Sample mean of X^2 is: ", meanXSquared]
|gib aus
Print["(Unbiased) sample variance of X is: ", varX]
|gib aus
Print["(Unbiased) sample variance of X^2 is: ", varXSquared]
|gib aus

Sample mean of X is: 0.995353
Sample mean of X^2 is: 1.98367
(Unbiased) sample variance of X is: 0.993044
(Unbiased) sample variance of X^2 is: 33.8872

```

```
In[27]:= (* Confidence interval for EX^2 *)
criticalTValue = Quantile[StudentTDistribution[n - 1], 0.975];
               |Quantil      |Student-t-Verteilung
meanXSquared - criticalTValue * varXSquared / Sqrt[n]
               |Quadratwurzel
meanXSquared + criticalTValue * varXSquared / Sqrt[n]
               |Quadratwurzel
```

Out[28]= 1.31942

Out[29]= 2.64793

```
In[30]:= (* Confidence interval for VarX *)
```

```
In[31]:= criticalChiSquaredValueLower = Quantile[ChiSquareDistribution[n - 1], 0.025];
               |Quantil      |Chi-Quadrat-Verteilung
criticalChiSquaredValueUpper = Quantile[ChiSquareDistribution[n - 1], 0.975];
               |Quantil      |Chi-Quadrat-Verteilung
varX * (n - 1) / criticalChiSquaredValueUpper
varX * (n - 1) / criticalChiSquaredValueLower
```

Out[33]= 0.966082

Out[34]= 1.02116

Exercise 5

```
In[1]:= (* Function that defines parameters and performs preliminary work *)
      |Funktion
preliminaries[] := Module[{},
      |Modul
      c = 2; (* Number of cranes *)
      |Zahl

      numberOfDays = 90; (* The number of days to be simulated *)

      (* Parameters for arrival and service distributions *)
      λ = 1.25; (* Mean interarrival time in days *)
      |arithmetisches Mittel

      meanServiceTime = 1;
      varianceOfServiceTime = 0.1;
      leftTruncation = 0.5;
      rightTruncation = 1.5;

      (* Definitions *)
      KINDARRIVAL = 1;
      KINDDEPARTURE = 2;
      KINDFASTDEPARTURE = 3;

      (* Statistical Counters *)
      totalNumberOfArrivals = 0;
      totalAmountOfCranesBusy = 0;
      totalWaitingTimeOfBoats = 0;
      longestWaitingTimeOfBoats = 0;

      (* Other parameters *)
      eventList = {}; (* Events in the list have
the form {timeOfOccurence, kind}, e.g. {5.2857, KINDDEPARTURE} *)
      queue = {}; (* Customers in the
queue have the form {time of arrival}, e.g. {3.4874} *)
      simulationClock = 0;
      currentNumberOfCranesBusy = 0;
];

In[2]:= (* Func draws from the Weibull distr. w/ mean λ *)
drawWeibullNumber[λ_] := -λ Log[RandomReal[]]
      |L... |reelle Zufallszahl
```



```

In[3]:= (* Func draws from N(meanServiceTime, varianceOfServiceTime)
      |numerischer Wert
      in [lowerTruncation, upperTruncation] using Box-Muller*)
      |Box
drawNormalNumber[meanServiceTime_, varianceOfServiceTime_,
  leftTruncation_, rightTruncation_] := Module[{sample},
      |Modul
  While[True,
    |solange|wahr
    (* Box-Muller method *)
    |Box
    sample = meanServiceTime + Sqrt[varianceOfServiceTime] *
      |Quadratwurzel
      Sqrt[-2 Log[RandomReal[]]] * Sin[2 * Pi * RandomReal[]];
      |Quadra... |L... |reelle Zufallszahl |Sinus |Kre...|reelle Zufallszahl
    (* Checks truncation *)
    If[leftTruncation < sample < rightTruncation,
      |wenn
      Break[];
      |beende Schleife
    ]
  ];

  sample
]

In[4]:= (* Function that adds an event to the event list *)
      |Funktion
addEventToEventList[event_] := Module[{},
      |Modul
      eventList = Sort[Append[eventList, event]];
      |sorti...|hänge an
    ];

In[5]:= (* Function that removes the next event from the event list and returns it *)
      |Funktion
removeAndReturnNextEventFromList[] := Module[{nextEvent},
      |Modul
      nextEvent = First[eventList];
      |erstes Element
      eventList = Drop[eventList, 1];
      |entferne Element
      nextEvent
    ];

In[6]:= (* Function that removes the next boat from the queue and returns it *)
      |Funktion
removeAndReturnNextBoatFromQueue[] := Module[{nextBoat},
      |Modul
      nextBoat = First[queue];
      |erstes Element
      queue = Drop[queue, 1];
      |entferne Element
      nextBoat
    ];

```

```

In[7]:= (* Function that starts the
[Funktion
simulation: schedules the first arrival to start up the event list *)
initialise[] := Module[{},
[Modul
    firstArrivalTime = drawWeibullNumber[λ];
    addEventToEventList[{firstArrivalTime, KINDARRIVAL}];
];

In[8]:= (* Function that processes an arrival event *)
[Funktion
handleArrivalEvent[arrivalTime_] := Module[{nextArrivalTime, serviceTime},
[Modul
    (* Handle the arrival *)
    totalNumberOfArrivals += 1;

    If[currentNumberOfCranesBusy < c, (* one or two cranes available *)
[wenn
        (* The arriving boat can be taken into service immediately *)
        serviceTime = drawNormalNumber[meanServiceTime,
            varianceOfServiceTime, leftTruncation, rightTruncation];

        If[currentNumberOfCranesBusy == 0,
[wenn
            (* Two cranes available *)
            totalWaitingTimeOfBoats += serviceTime/2;
            longestWaitingTimeOfBoats = Max[longestWaitingTimeOfBoats, serviceTime/2];
[größtes Element
            addEventToEventList[{simulationClock + serviceTime/2, KINDFASTDEPARTURE}];
            currentNumberOfCranesBusy += 2;
            ,

            (* One crane available *)
            totalWaitingTimeOfBoats += serviceTime;
            longestWaitingTimeOfBoats = Max[longestWaitingTimeOfBoats, serviceTime];
[größtes Element
            addEventToEventList[{simulationClock + serviceTime, KINDDEPARTURE}];
            currentNumberOfCranesBusy += 1;
        ],

        (* The arriving boat must queue *)
        queue = Append[queue, arrivalTime];
[hänge an
    ];

    (* Now that the arrival is handled, schedule the next arrival *)
[jetzt
    nextArrivalTime = simulationClock + drawWeibullNumber[λ];
    addEventToEventList[{nextArrivalTime, KINDARRIVAL}];
];

```

```

In[9]:= (* Function that processes a departure event *)
      |Funktion
      handleDepartureEvent[departureTime_] := Module[{nextServiceTime},
      |Modul
      (* Check whether there is a boat queued up. *)
      |prüfe
      If[Length[queue] > 0,
      |... |Länge
      (* There is a boat
      queued: put this boat into service and schedule departure *)
      nextServiceTime = drawNormalNumber[meanServiceTime,
      varianceOfServiceTime, leftTruncation, rightTruncation];

      arrivalTimeOfBoatToBePutIntoService = removeAndReturnNextBoatFromQueue[];
      waitingTimeOfBoat =
      (simulationClock - arrivalTimeOfBoatToBePutIntoService) + nextServiceTime;
      totalWaitingTimeOfBoats += waitingTimeOfBoat;
      longestWaitingTimeOfBoats = Max[longestWaitingTimeOfBoats, waitingTimeOfBoat];
      |größtes Element
      addEventToEventList[{simulationClock + nextServiceTime, KINDDEPARTURE}];
      ,

      (* There is no customer queued: make the clerk available again *)
      currentNumberOfCranesBusy -= 1;
      ];
];

```

```

In[10]:= (* Function that processes a fast departure event *)
          [Funktion
handleFastDepartureEvent[departureTime_] := Module[{nextServiceTime},
          [Modul

(* Checks whether there is a boat queued up *)
If[Length[queue] > 0,
  [...] [Länge
  (* If there is at least one boat in the queue,
    [wenn
  the boat is scheduled with normal service time even though two berths are
    free after the fast departure of one boat unloaded with two cranes *)
  nextServiceTime = drawNormalNumber[meanServiceTime,
    varianceOfServiceTime, leftTruncation, rightTruncation];

  arrivalTimeOfBoatToBePutIntoService = removeAndReturnNextBoatFromQueue[];
  waitingTimeOfBoat =
    (simulationClock - arrivalTimeOfBoatToBePutIntoService) + nextServiceTime;
  totalWaitingTimeOfBoats += waitingTimeOfBoat;
  longestWaitingTimeOfBoats = Max[longestWaitingTimeOfBoats, waitingTimeOfBoat];
  [größtes Element
  addEventToEventList[{simulationClock + nextServiceTime, KINDEPARTURE}];

(* Check whether there is another boat queued up *)
  [prüfe
  If[Length[queue] > 0,
    [...] [Länge
    (* There is another boat
      queued: put it into service and schedule departure *)
    nextServiceTime = drawNormalNumber[meanServiceTime,
      varianceOfServiceTime, leftTruncation, rightTruncation];

    arrivalTimeOfBoatToBePutIntoService = removeAndReturnNextBoatFromQueue[];
    waitingTimeOfBoat =
      (simulationClock - arrivalTimeOfBoatToBePutIntoService) + nextServiceTime;
    totalWaitingTimeOfBoats += waitingTimeOfBoat;
    longestWaitingTimeOfBoats =
      Max[longestWaitingTimeOfBoats, waitingTimeOfBoat];
    [größtes Element
    addEventToEventList[{simulationClock + nextServiceTime, KINDEPARTURE}];

    ,
    currentNumberOfCranesBusy -= 1;
  ];
  ,

(* There is no boat queued: make both cranes available again *)
  currentNumberOfCranesBusy -= 2;
];
];

```

```

In[11]:= (* Function that takes the next event from the event list and processes it. *)
      (*Funktion
handleNextEvent[] := Module[{nextEvent, eventTime, eventKind},
      (*Modul
      nextEvent = removeAndReturnNextEventFromList[];
      eventTime = nextEvent[[1]];
      eventKind = nextEvent[[2]];

      (* Update counters and parameters *)
      (*aktualisiere
      simulationClock = eventTime;

      (* Checks whether the next event
      is an arrival or a departure and calls functions accordingly *)
      If[eventKind == KINDARRIVAL,
      (*wenn
      handleArrivalEvent[eventTime];
      ,
      If[eventKind == KINDDEPARTURE,
      (*wenn
      handleDepartureEvent[eventTime];,

      handleFastDepartureEvent[eventTime];
      ];
      ];
];

```

```

In[12]:= (* Function that reports the input parameters
[Funktion
and the findings of the simulation program *)
report[] := Module[{
[Modul

    Print["====="];
    [gib aus
    Print["== Simulation run of a harbor example =="];
    [gib aus
    Print["====="];
    [gib aus
    Print["Input parameters:"];
    [gib aus [Eingabe
    Print["Number of cranes c equals ", c, "."];
    [gib aus [Zahl

    Print["Mean interarrival time  $\lambda$  equals ",  $\lambda$ , " per day."];
    [gib aus [arithmetisches Mittel

    Print["Lower bound of service time ~ N(1,0.1) is ",
    [gib aus [numerischer Wert
    leftTruncation, " day(s)."];
    Print["Upper bound of service time N(1,0.1) is ",
    [gib aus [numerischer Wert
    rightTruncation, " day(s)."];
    Print["====="];
    [gib aus
    Print["Results:"];
    [gib aus

    Print["Duration of the simulation is ", simulationClock, " days"];
    [gib aus [Dauer

    Print["The total number of boats that arrived equals ",
    [gib aus
    totalNumberOfArrivals, "."];
    Print["The average waiting time of boats equals ",
    [gib aus
    totalWaitingTimeOfBoats / totalNumberOfArrivals, " day(s)."];
    Print["The longest waiting time of boats equals ",
    [gib aus
    longestWaitingTimeOfBoats, " day(s)."];
    ];

```

```

In[13]:= (* Main Function *)
          [Funktion
runProgram[] := Module[{averageWaitingTime},
          [Modul
                preliminaries[];
                initialise[];

                (* Handle events when the time of the earliest upcoming event,
                eventList[[1]][[1]], occurs before the end of the day,
                marked by numberOfHoursInADay *)
                While[eventList[[1]][[1]] < numberOfDays,
                [solange

                        handleNextEvent[];

                ];
                report[];

                averageWaitingTime = totalWaitingTimeOfBoats / totalNumberOfArrivals;
                {averageWaitingTime, longestWaitingTimeOfBoats}
                ];

In[14]:= (* Actually run the program: *)
runProgram[];

=====
=== Simulation run of a harbor example ===
=====

Input parameters:
Number of cranes c equals 2.
Mean interarrival time  $\lambda$  equals 1.25 per day.
Lower bound of service time  $\sim N(1,0.1)$  is 0.5 day(s).
Upper bound of service time  $N(1,0.1)$  is 1.5 day(s).

=====

Results:
Duration of the simulation is 89.9639 days
The total number of boats that arrived equals 61.
The average waiting time of boats equals 0.774873 day(s).
The longest waiting time of boats equals 1.81317 day(s).

In[15]:= (* Checks result consistency by replication to check simulation reliability*)
n = 100;
simulationResults = Table[runProgram[], {i, n}];
          [Tabelle

```

In[17]:=

```

(*Extract the first and second elements from each sub-list*)
extrahiere
averageWaitingTimes = First /@ simulationResults;
erstes Element
longestWaitingTimes = simulationResults[[All, 2]];
alle

(*Calculate the average of the first elements*)
averageOfAverageWaitingTimes = Mean[averageWaitingTimes]
arithmetisches Mittel
varianceOfAverageWaitingTimes = Variance[averageWaitingTimes]
Varianz

averageOfLongestWaitingTimes = Mean[longestWaitingTimes]
arithmetisches Mittel
varianceOfLongestWaitingTimes = Variance[longestWaitingTimes]
Varianz

```

Out[19]= 0.849638

Out[20]= 0.00730321

Out[21]= 2.15093

Out[22]= 0.132015